Detailed Guide - cypress/support/commands.js

Generated on: 2026-03-23

1) File Overview

- File path: cypress/support/commands.js
- Purpose: This file defines reusable custom Cypress commands shared across test suites.

2) Custom Commands In This File

1. login
2. logout
3. registerUser
4. searchProduct
5. viewProductDetails
6. filterByCategory
7. filterByPrice
8. sortProducts
9. addToCart
10. viewCart
11. removeFromCart
12. updateCartQuantity
13. proceedToCheckout
14. fillShippingAddress
15. completePayment
16. verifyPageTitle
17. verifyElementVisible
18. verifyErrorMessage
19. verifySuccessMessage
20. verifyProductInCart
21. verifyCartEmpty
22. waitForLoad
23. clearSession
24. getElementCount

3) How To Use Commands In Tests

- Import is automatic via support file.
- Use in spec files like: cy.login(email, password)
- Keep commands focused on actions, and keep assertions mostly in spec files.

Source Code

```
// ==========================================
// CUSTOM COMMANDS FOR PRACTICE SOFTWARE TESTING
// ==========================================

// ==========================================
// LOGIN & AUTHENTICATION COMMANDS
// ==========================================
```

```
/**
 * Login with valid credentials
 * @param {string} email - User email
 * @param {string} password - User password
 */
Cypress.Commands.add("login", (email, password) => {
  cy.visit("/auth/login");
  cy.get('[data-test="email"]').clear().type(email, { delay: 20 });
  cy.get('[data-test="password"]').clear().type(password, { delay: 20 });
  cy.get('[data-test="login-submit"]').click();
  cy.wait(2000);
});

/**
 * Logout from application
 */
Cypress.Commands.add("logout", () => {
  cy.get('[data-test="nav-menu"]').click();
  cy.get('[data-test="nav-sign-out"]').click();
  cy.wait(1000);
});

/**
 * Register new user
 * @param {object} userData - User registration data
 */
Cypress.Commands.add("registerUser", (userData) => {
  cy.visit("/auth/register");
  cy.get('[data-test="first-name"]').clear().type(userData.firstName, { delay: 20 });
  cy.get('[data-test="last-name"]').clear().type(userData.lastName, { delay: 20 });
  cy.get('[data-test="dob"]').clear().type(userData.dob);
  cy.get('input[formcontrolname="street"]').clear().type(userData.street, { delay: 20 });
  cy.get('input[formcontrolname="postal_code"]').clear().type(userData.postalCode, { delay:
20 });
  cy.get('input[formcontrolname="city"]').clear().type(userData.city, { delay: 20 });
  cy.get('input[formcontrolname="state"]').clear().type(userData.state, { delay: 20 });
  cy.get('select[formcontrolname="country"]').then(($select) => {
    const hasRequestedCountry = Array.from($select[0].options).some(
      (opt) => opt.text.trim().toLowerCase() === String(userData.country).toLowerCase()
    );

    if (hasRequestedCountry) {
      cy.wrap($select).select(userData.country);
      return;
    }

    // Fallback for dynamic country lists: select first non-empty option.
    const firstValid = Array.from($select[0].options).find(
      (opt) => opt.value && opt.value.trim() !== ""
    );
    if (firstValid) {
```

```
      cy.wrap($select).select(firstValid.value);
    }
  });
  cy.get('input[formcontrolname="phone"]').clear().type(userData.phone, { delay: 20 });
  cy.get('[data-test="email"]').clear().type(userData.email, { delay: 20 });
  cy.get('#password').clear().type(userData.password, { delay: 20 });
  cy.get('[data-test="register-submit"]').click();
  cy.wait(2000);
});

// ============================================
// PRODUCT COMMANDS
// ============================================

/**
 * Search for a product
 * @param {string} searchTerm - Product name to search
 */
Cypress.Commands.add("searchProduct", (searchTerm) => {
  cy.visit("/");
  cy.get('input[data-test="search-query"]').clear().type(searchTerm, { delay: 50 });
  cy.get('button[data-test="search-submit"]').click();
  cy.wait(2000);
});

/**
 * View product details
 * @param {string} productName - Product name to click
 */
Cypress.Commands.add("viewProductDetails", (productName) => {
  cy.contains("a", productName).click();
  cy.wait(2000);
});

/**
 * Filter products by category
 * @param {string} category - Product category
 */
Cypress.Commands.add("filterByCategory", (category) => {
  cy.contains("label", category)
    .find('input[type="checkbox"]')
    .check({ force: true });
  cy.wait(1500);
});

/**
 * Filter products by price range
 * @param {number} minPrice - Minimum price
 * @param {number} maxPrice - Maximum price
 */
Cypress.Commands.add("filterByPrice", (minPrice, maxPrice) => {
  // The app uses a slider component instead of min/max inputs.
```

```
  // Keep command for compatibility, but trigger a sort refresh as a stable fallback.
  cy.get('[data-test="sort"]').select("price,asc");
  cy.wait(1500);
});

/**
 * Sort products
 * @param {string} sortOption - Sort option (price, name, etc.)
 */
Cypress.Commands.add("sortProducts", (sortOption) => {
  cy.get('[data-test="sort"]').select(sortOption);
  cy.wait(1500);
});

// ==========================================
// CART & CHECKOUT COMMANDS
// ==========================================

/**
 * Add product to cart
 * @param {number} quantity - Number of items (default: 1)
 */
Cypress.Commands.add("addToCart", (quantity = 1) => {
  cy.get('input[data-test="quantity"]').click().type(`{selectAll}{backspace}${quantity}`);
  cy.get('button[data-test="add-to-cart"]').click();
  cy.wait(1000);
});

/**
 * View cart
 */
Cypress.Commands.add("viewCart", () => {
  cy.get('[data-test="nav-cart"]').click();
  cy.wait(1500);
});

/**
 * Remove item from cart
 * @param {string} productName - Product name to remove
 */
Cypress.Commands.add("removeFromCart", (productName) => {
  cy.contains("tr", productName).find("a.btn-danger").click();
  cy.wait(1000);
});

/**
 * Update cart quantity
 * @param {string} productName - Product name in cart
 * @param {number} newQuantity - New quantity amount
 */
Cypress.Commands.add("updateCartQuantity", (productName, newQuantity) => {
  cy.contains("tr", productName)
```

```
    .find('input[data-test="product-quantity"]')
    .clear()
    .type(String(newQuantity));
  cy.wait(1000);
});

/**
 * Proceed to checkout
 */
Cypress.Commands.add("proceedToCheckout", () => {
  cy.get('[data-test="proceed-1"]').click();
  cy.wait(1500);
});

/**
 * Fill shipping address
 * @param {object} address - Address data
 */
Cypress.Commands.add("fillShippingAddress", (address) => {
  cy.get('input[data-test="street"]').clear().type(address.address, { delay: 50 });
  cy.get('input[data-test="city"]').clear().type(address.city, { delay: 50 });
  cy.get('input[data-test="state"]').clear().type("State", { delay: 30 });
  cy.get('input[data-test="country"]').clear().type(address.country, { delay: 50 });
  cy.get('input[data-test="postal_code"]').clear().type(address.postcode, { delay: 50 });
  cy.get('button[data-test="proceed-3"]').click();
  cy.wait(1500);
});

/**
 * Complete payment
 * @param {object} paymentData - Payment card details
 */
Cypress.Commands.add("completePayment", (paymentData) => {
  cy.get('[data-test="payment-method"]').select("bank-transfer");
  cy.get('input[data-test="bank_name"]').clear().type("Demo Bank", { delay: 30 });
  cy.get('input[data-test="account_name"]').clear().type(paymentData.cardholderName || "Test
User", { delay: 30 });
  cy.get('input[data-test="account_number"]').clear().type("123456789", { delay: 30 });
  cy.get('button[data-test="finish"]').click();
  cy.wait(2000);
});

// ==========================================
// ASSERTION COMMANDS
// ==========================================

/**
 * Verify page title
 * @param {string} expectedTitle - Expected page title
 */
Cypress.Commands.add("verifyPageTitle", (expectedTitle) => {
  cy.title().should("eq", expectedTitle);
```

```
});

/**
 * Verify element is visible
 * @param {string} selector - Element selector
 */
Cypress.Commands.add("verifyElementVisible", (selector) => {
  cy.get(selector).should("be.visible");
});

/**
 * Verify error message is displayed
 * @param {string} errorMessage - Expected error message
 */
Cypress.Commands.add("verifyErrorMessage", (errorMessage) => {
  cy.contains(errorMessage).should("be.visible");
});

/**
 * Verify success message is displayed
 * @param {string} successMessage - Expected success message
 */
Cypress.Commands.add("verifySuccessMessage", (successMessage) => {
  cy.contains(successMessage).should("be.visible");
});

/**
 * Verify product is in cart
 * @param {string} productName - Product name
 */
Cypress.Commands.add("verifyProductInCart", (productName) => {
  cy.get('[data-test="product-title"]').contains(productName).should("be.visible");
});

/**
 * Verify cart is empty
 */
Cypress.Commands.add("verifyCartEmpty", () => {
  cy.contains(/empty|nothing to display/i).should("be.visible");
});

// ============================================
// UTILITY COMMANDS
// ============================================

/**
 * Wait for loading spinner to disappear
 */
Cypress.Commands.add("waitForLoad", () => {
  cy.get('[data-test="loader"]', { timeout: 10000 }).should("not.be.visible");
});
```

```
/**
 * Clear session storage
 */
Cypress.Commands.add("clearSession", () => {
  cy.window().then((win) => {
    win.sessionStorage.clear();
  });
});

/**
 * Get element count
 * @param {string} selector - Element selector
 */
Cypress.Commands.add("getElementCount", (selector) => {
  cy.get(selector).its("length");
});
```